

Top 50 Tricky SQL Theoretical Interview Questions

1. What is the difference between the **WHERE** clause and the **HAVING** clause, and when should each be used?

- **WHERE**: Filters rows BEFORE aggregation. Works on individual row data. Cannot use aggregate functions directly.
- **HAVING**: Filters groups AFTER aggregation. Works on aggregated data (SUM, COUNT, AVG, etc.).

Example:

```
SELECT department, AVG(salary)
FROM Employee
WHERE salary > 30000
GROUP BY department
HAVING AVG(salary) > 50000
```

Execution Order: WHERE → GROUP BY → HAVING

2. What are the differences between **DELETE**, **TRUNCATE**, and **DROP** commands in SQL?

Feature	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
WHERE clause	Yes	No	N/A
Speed	Slow	Fast	Fastest
Rollback	Yes	No (varies by DB)	No
Space	Retains	Releases	Removes all
Triggers	Fires	Doesn't fire	N/A
Identity	Not reset	Reset to seed	N/A

When to use: DELETE for specific rows, TRUNCATE for clearing all data, DROP for removing table structure.

3. Why does SQL treat NULL differently from other values, and what does it actually represent?

NULL Represents: Unknown or missing value, not zero or empty string.

Three-Valued Logic:

- SQL uses TRUE, FALSE, and UNKNOWN (NULL)
- NULL propagates through operations: $\text{NULL} + 5 = \text{NULL}$, $\text{NULL} * 0 = \text{NULL}$
- Comparisons with NULL return UNKNOWN, not TRUE or FALSE

Why Different:

- We don't know what NULL is, so we can't compare it definitively
- NULL in WHERE clause filters out the row (UNKNOWN is treated as FALSE)
- Use `IS NULL` or `IS NOT NULL` for NULL checks, never `=`

Example:

```
SELECT * FROM Employee WHERE salary = NULL; -- Wrong, returns 0 rows
SELECT * FROM Employee WHERE salary IS NULL; -- Correct, returns NULL salaries
```

4. Why does the expression `NULL = NULL` not evaluate to `TRUE` in SQL?

Reason: Since NULL means "unknown", comparing two unknowns doesn't give a definitive TRUE. Instead, it returns UNKNOWN.

Three-Valued Logic:

- `UNKNOWN = UNKNOWN` → UNKNOWN (not TRUE)
- `UNKNOWN > 5` → UNKNOWN
- `NULL AND TRUE` → UNKNOWN
- `NULL OR TRUE` → TRUE (UNKNOWN OR TRUE is always TRUE)

Correct NULL Comparison:

```
SELECT * FROM Employee WHERE NULL = NULL; -- Returns 0 rows
SELECT * FROM Employee WHERE NULL IS NULL; -- Returns all rows with NULL
```

Key Point: Always use `IS NULL` or `IS NOT NULL` for NULL checks.

5. What is the difference between `COUNT(*)`, `COUNT(column_name)`, and `COUNT(DISTINCT column_name)`?

COUNT(*) - Counts all rows including NULL

- Returns total number of rows in result set
- Fastest (no column checking)
- Example: `COUNT(*) = 100`

COUNT(column_name) - Counts non-NULL values in column

- Ignores NULL values
- Slightly slower than COUNT(*)
- Example: `COUNT(manager_id) = 95` (5 NULLs excluded)

COUNT(DISTINCT column_name) - Counts unique non-NULL values

- Removes duplicates and ignores NULL
- Slowest (requires sorting/hashing)
- Example: `COUNT(DISTINCT department) = 10`

Table Comparison:

```
100 employees, 5 with NULL manager_id, 10 departments, 45 unique managers
COUNT(*)                = 100
COUNT(manager_id)       = 95
COUNT(DISTINCT manager_id) = 45
COUNT(DISTINCT department) = 10
```

6. What is the difference between CHAR and VARCHAR, and when would you choose one over the other?

Feature	CHAR	VARCHAR
Storage	Fixed length (pads with spaces)	Variable length
Space	Wastes space if shorter	Efficient storage
Performance	Slightly faster	Slightly slower
Best For	Fixed-length data	Variable-length data

Use CHAR for:

- Fixed identifiers: Country codes (US), Status (A/I), Postal codes
- Performance-critical fields: Always same length
- Example: `CHAR(2)` for state abbreviations

Use VARCHAR for:

- Names, addresses, email, descriptions
- Variable-length input
- Example: `VARCHAR(100)` for employee names

Storage Example: `CHAR(10): "ABC"` stored as "ABC " (10 bytes) `VARCHAR(10): "ABC"` stored as "ABC" (~5 bytes)

7. What is the difference between a primary key, a unique key, and a foreign key?

Aspect	Primary Key	Unique Key	Foreign Key
Purpose	Uniquely identifies row	Ensures uniqueness	Links to another table
NULL Allowed	NO	YES	Usually YES
Per Table	ONE only	MULTIPLE	MULTIPLE
Index	Clustered (usually)	Non-clustered	No automatic
Referential Integrity	No	No	YES
Example	EmployeeID	Email, SSN	DepartmentID

Implementation:

```
CREATE TABLE Employee (  
    EmployeeID INT PRIMARY KEY,  
    Email VARCHAR(100) UNIQUE,  
    SSN VARCHAR(11) UNIQUE NOT NULL,  
    DepartmentID INT,  
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)  
);
```

8. Can a table have multiple unique keys and multiple foreign keys, and why?

YES to both:

Multiple Unique Keys (unlimited):

- One PRIMARY KEY only
- Many UNIQUE KEYS allowed
- Each identifies rows differently
- Example: Email, SSN, EmployeeCode all unique

Multiple Foreign Keys (unlimited):

- Establish relationships with different tables
- Example: DepartmentID, ManagerID, ProjectID

Example:

```
CREATE TABLE Employee (  
    EmployeeID INT PRIMARY KEY,  
    Email VARCHAR(100) UNIQUE,           -- Unique Key 1  
    SSN VARCHAR(11) UNIQUE,             -- Unique Key 2  
    DepartmentID INT,                   -- Foreign Key 1  
    ManagerID INT,                      -- Foreign Key 2  
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID),  
    FOREIGN KEY (ManagerID) REFERENCES Employee(EmployeeID)  
);
```

Why: Supports complex relationships and multiple unique identifiers.

9. What are candidate keys and alternate keys, and how are they different from primary keys?

Candidate Key: Any column(s) that could uniquely identify each row (must be unique, NOT NULL)

Alternate Key: A candidate key that is NOT the primary key (secondary identifier)

Primary Key: ONE candidate key chosen as main identifier

Example:

```
Employee Table Candidate Keys:
- EmployeeID (chosen as PRIMARY KEY)
- Email (alternate key - unique, not NULL)
- SSN (alternate key - unique, not NULL)

CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE NOT NULL,    -- Alternate Key 1
    SSN VARCHAR(11) UNIQUE NOT NULL        -- Alternate Key 2
);
```

Relationship: Primary Key $\subset$ Candidate Keys. All remaining candidate keys = Alternate Keys.

10. Why is SQL known as a declarative language instead of a procedural language?

Declarative (SQL):

- Specify WHAT you want, not HOW
- DBMS decides execution strategy
- Example: `SELECT * FROM Employee WHERE salary > 50000;`

Procedural (Java, C, Python):

- Specify HOW to accomplish task
- Developer controls step-by-step operations
- Example: Loop through array, compare each salary, add to results

Advantages of Declarative SQL:

1. Simpler to write and understand
2. Query optimizer finds best execution path
3. Portable across different databases
4. Less error-prone (no manual loops)
5. Easier maintenance and future optimizations

Comparison:

Aspect	SQL (Declarative)	Java (Procedural)
Complexity	Simple	Complex
Optimization	Automatic	Manual
Performance	DB optimized	Dev responsible
Maintainability	Easy	Hard

Joins

11. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

INNER JOIN: Only matching rows from both tables

- Excludes unmatched rows from both sides

LEFT JOIN: All rows from left table + matching rows from right

- Right table columns show NULL if no match

RIGHT JOIN: All rows from right table + matching rows from left

- Left table columns show NULL if no match

FULL OUTER JOIN: All rows from both tables

- Unmatched columns show NULL from both sides
- (Not available in MySQL; use UNION)

Visual:

```

INNER JOIN: [===A==B===] Only overlapping
LEFT JOIN:  [===A==B===] All of A, overlap from B
RIGHT JOIN: [===A==B===] All of B, overlap from A
FULL JOIN:  [===A==B===] All from both

```

12. Why do joins sometimes produce duplicate rows even when the data appears correct?

Cause: Multiple matches in join condition create Cartesian product.

Example:

```

Customers:           Orders (Customer 1 has 3 orders):
ID  Name             CustomerID  OrderID
1   John             1           1
2   Jane             1           2
                1           3

```

JOIN Result (3 rows - one for each order):

```

1  John  1
1  John  2
1  John  3

```

When Multiple Matches Occur:

- Right table has multiple rows with same foreign key
- Left table has multiple rows with same join key
- Both tables have multiple matches (m-to-n relationship)

Prevention:

1. Verify join key cardinality (is it supposed to be unique?)

2. Add DISTINCT if duplicates acceptable: `SELECT DISTINCT *`
3. Use GROUP BY if aggregation needed
4. Check if join condition is correct
5. Add constraint to ensure uniqueness

13. What is the difference between EXISTS and INNER JOIN, and when is one preferred over the other?

EXISTS (Subquery):

- Checks IF matching rows exist (TRUE/FALSE)
- Stops after finding first match
- No duplicate rows
- Returns only outer query columns
- Semi-join semantics

INNER JOIN:

- Matches and returns rows from both tables
- Processes ALL matching rows
- Can produce duplicates
- Returns columns from both tables

Performance Comparison:

Factor	EXISTS	INNER JOIN
Search	Stops at first match	All matches
Duplicates	No	Possible
Columns	Outer only	Both tables
Large datasets	Faster	Slower
Small datasets	Similar	Similar

Use EXISTS when: Checking existence, need outer columns only, performance critical

Use INNER JOIN when: Need both tables' columns, want explicit results, aggregation needed

14. What is the difference between LEFT JOIN with IS NULL and NOT EXISTS for finding unmatched records?

LEFT JOIN with IS NULL:

```
SELECT c.* FROM Customers c
LEFT JOIN Orders o ON c.ID = o.CustomerID
WHERE o.CustomerID IS NULL;
```

- Returns left table rows without matches
- Produces NULL for unmatched right columns
- Works on any database

NOT EXISTS:

```
SELECT c.* FROM Customers c
WHERE NOT EXISTS (
    SELECT 1 FROM Orders o
    WHERE o.CustomerID = c.ID
);
```

- Checks absence of matches
- Stops searching after finding first match (better performance)
- Semi-join semantics (no duplicate issue)

Differences:

Aspect	LEFT JOIN + IS NULL	NOT EXISTS
Performance	Slower for large datasets	Faster (stops at first match)
Duplicates	Possible if join key not unique	Never
Readability	Intuitive	Explicit checking
NULL handling	Issues if join key is NULL	Cleaner
Best Use	Simple cases	Large datasets

Recommendation: NOT EXISTS is generally preferred for better performance.

15. Under what circumstances can a Cartesian product occur, and how can it be prevented?

Cartesian Product Occurs When:

1. JOIN without ON condition: `FROM A JOIN B` (no ON clause)
2. Incorrect join condition: `ON 1=1` or `ON TRUE`
3. Missing join condition between multiple tables
4. Multiple tables without proper linkage

Example:

```
-- WRONG: Cartesian product
SELECT * FROM Customers, Orders; -- Every customer matched with every order
RESULT: 100 customers * 500 orders = 50,000 rows

-- CORRECT: Proper join condition
SELECT * FROM Customers
JOIN Orders ON Customers.ID = Orders.CustomerID;
RESULT: Matching pairs only (~300 rows)
```

Prevention:

1. Always specify ON condition in JOINS

2. Link tables via foreign keys
3. Verify join logic before execution
4. Use WHERE clause to filter if needed
5. Test with small datasets first
6. Check execution plan

Anti-pattern to avoid:

```
-- BAD: Cartesian product
SELECT * FROM A, B, C; -- Returns |A| * |B| * |C| rows

-- GOOD: Explicit joins with conditions
SELECT * FROM A
JOIN B ON A.id = B.a_id
JOIN C ON B.id = C.b_id;
```

16. Why is it important to specify the correct join condition while joining multiple tables?

Importance:

1. **Correctness:** Wrong condition returns incorrect data
2. **Performance:** Proper join eliminates unnecessary Cartesian products
3. **Duplicate Prevention:** Correct join keys avoid duplicate rows
4. **Data Integrity:** Ensures logical relationships are maintained
5. **Business Logic:** Reflects actual data relationships

Example of Wrong vs Right:

```
-- WRONG: No condition (Cartesian product)
SELECT * FROM Customers, Orders; -- 100 * 500 = 50,000 rows

-- WRONG: Incorrect condition
SELECT * FROM Customers
JOIN Orders ON Customers.Name = Orders.CustomerName; -- May miss matches

-- CORRECT: Proper foreign key relationship
SELECT * FROM Customers
JOIN Orders ON Customers.ID = Orders.CustomerID; -- Correct cardinality
```

Impact:

- Wrong condition → Wrong results, performance issues, incorrect business decisions
- Right condition → Accurate data, optimal performance, correct insights

17. Can a query contain multiple joins to the same table, and what are common use cases for doing so?

YES: A query can JOIN the same table multiple times (using aliases).

Common Use Cases:

1. Organizational Hierarchy:

```
SELECT e.Name AS Employee, m.Name AS Manager, g.Name AS GrandManager
FROM Employee e
LEFT JOIN Employee m ON e.ManagerID = m.EmployeeID
LEFT JOIN Employee g ON m.ManagerID = g.EmployeeID;
```

2. Comparative Analysis:

```
SELECT e1.Name, e1.Salary, e2.Name, e2.Salary
FROM Employee e1
JOIN Employee e2 ON e1.DepartmentID = e2.DepartmentID
WHERE e1.Salary > e2.Salary;
```

3. Self-referential Data:

```
SELECT * FROM Product p1
JOIN Product p2 ON p1.ParentID = p2.ProductID;
```

Key Point: Use aliases to differentiate same table references: `e, m, g, e1, e2`

18. What is a self join, and in what situations is it commonly used?

Self Join: Joining a table with itself (using different aliases) to compare rows within the same table.

Common Situations:

1. Reporting Hierarchy:

```
SELECT e.EmployeeID, e.Name AS Employee,
       m.EmployeeID, m.Name AS Manager
FROM Employee e
LEFT JOIN Employee m ON e.ManagerID = m.EmployeeID;
```

2. Finding Duplicates:

```
SELECT DISTINCT e1.* FROM Employee e1
JOIN Employee e2 ON e1.Email = e2.Email
WHERE e1.EmployeeID < e2.EmployeeID; -- Avoid duplicate pairs
```

3. Comparing Salaries:

```
SELECT e1.Name, e1.Salary, e2.Name, e2.Salary
FROM Employee e1
JOIN Employee e2 ON e1.DepartmentID = e2.DepartmentID
WHERE e1.Salary < e2.Salary;
```

4. Component Assembly:

```
SELECT parent.ComponentName, child.ComponentName
FROM Component parent
JOIN Component child ON parent.ComponentID = child.ParentID;
```

GROUP BY and Aggregation

19. Why must every non-aggregated column in the **SELECT** list appear in the **GROUP BY** clause?

Reason: Without GROUP BY requirement, the database wouldn't know which row value to return when multiple rows are grouped.

Example of Ambiguity:

```
-- WRONG: Which EmployeeName? (multiple employees per department)
SELECT Department, EmployeeName, COUNT(*) AS EmployeeCount
FROM Employee
GROUP BY Department;

-- CORRECT: All non-aggregated columns in GROUP BY
SELECT Department, EmployeeName, COUNT(*) AS EmployeeCount
FROM Employee
GROUP BY Department, EmployeeName;
```

The Logic:

- GROUP BY creates groups of rows
- Aggregate functions (SUM, COUNT, AVG) work on each group
- Non-aggregated columns must either:
 1. Be in GROUP BY (consistent within each group), OR
 2. Be wrapped in an aggregate function

Valid Examples:

```
-- Valid: All non-aggregated columns in GROUP BY
SELECT Department, AVG(Salary) FROM Employee GROUP BY Department;

-- Valid: Aggregate the non-grouped column
SELECT Department, MAX(EmployeeName), AVG(Salary) FROM Employee GROUP BY
Department;
```

20. What is the difference between **GROUP BY** and **DISTINCT**, even though both remove duplicate values?

GROUP BY:

- Groups rows, enables aggregation functions (SUM, COUNT, AVG)
- Can include aggregate functions in SELECT
- Allows filtering with HAVING
- More processing (grouping logic)
- Use when you need aggregated data

DISTINCT:

- Removes duplicate rows from result
- Cannot use aggregate functions
- No grouping logic
- Less processing overhead
- Use when you need unique rows only

Example:

```
-- Same Result (if no aggregation):
SELECT DISTINCT Department FROM Employee;
SELECT Department FROM Employee GROUP BY Department;

-- DIFFERENT Results (GROUP BY enables aggregation):
SELECT DISTINCT Department FROM Employee; -- Just departments
SELECT Department, COUNT(*) FROM Employee GROUP BY Department; -- Count per dept
```

Performance:

- **DISTINCT**: Often faster for simple deduplication
- **GROUP BY**: Slower but provides aggregation capabilities

Use DISTINCT when: Need unique rows **Use GROUP BY when:** Need unique rows AND aggregation data

21. How does SQL handle **NULL** values while performing aggregate functions?

General Rule: Most aggregate functions ignore NULL values.

By Function:

- **COUNT(*)**: Counts all rows including NULL → Counts NULLs
- **COUNT(column)**: Counts non-NULL values → Ignores NULLs
- **SUM()**: Ignores NULL → NULL values excluded
- **AVG()**: Ignores NULL → Calculated on non-NULL only
- **MIN()/MAX()**: Ignores NULL → Finds min/max of non-NULL
- **STDDEV()/VARIANCE()**: Ignores NULL

Example:

Employee Salary: 50000, 60000, NULL, 70000

COUNT(*)	= 4	(all rows)
COUNT(salary)	= 3	(NULL excluded)
SUM(salary)	= 180000	(NULL ignored)
AVG(salary)	= 60000	(180000/3, not 180000/4)
MIN(salary)	= 50000	
MAX(salary)	= 70000	

Handling NULLs in Aggregation:

```
-- COALESCE: Replace NULL with default value
SELECT AVG(COALESCE(Salary, 0)) FROM Employee;

-- COUNT NULL values:
SELECT COUNT(*) - COUNT(Salary) AS NullCount FROM Employee;
```

Important: NULL propagates through calculations, returns NULL in most cases.

22. What happens when aggregate functions are used without a GROUP BY clause?

Behavior: Aggregate function operates on ENTIRE result set, returns single row.

Example:

Employee Salary: 50000, 60000, 70000, 80000

```
-- Without GROUP BY (operates on all 4 rows):
SELECT COUNT(*) AS TotalEmployees, AVG(Salary) AS AvgSalary
FROM Employee;
```

RESULT: 1 row

TotalEmployees	AvgSalary
4	65000

```
-- With GROUP BY (operates on each group):
SELECT Department, COUNT(*), AVG(Salary)
FROM Employee
GROUP BY Department;
```

RESULT: Multiple rows (one per department)

Key Points:

1. Without GROUP BY → Single aggregated result
2. With GROUP BY → Aggregated result per group
3. Mixing non-aggregated columns without GROUP BY causes error

Invalid Query:

```
-- ERROR: Department is non-aggregated but no GROUP BY
SELECT Department, COUNT(*) FROM Employee; -- Which department to show?
```

23. Why does `COUNT(column_name)` ignore `NULL` values while `COUNT(*)` counts every row?

COUNT(column_name): Counts non-NULL occurrences of that column

- Designed to count existing values
- NULL means "no value", so not counted
- Returns count of non-NULL entries

COUNT(*): Counts all rows including those with any NULLs

- Doesn't check individual columns
- Counts the existence of rows
- Returns total row count

Example:

Employee Table:

ID	Name	Salary	ManagerID	
1	John	50000	101	
2	Jane	60000	NULL	(no manager)
3	Bob	NULL	101	(no salary)
4	Alice	70000	102	

```
COUNT(*)           = 4 (all rows)
COUNT(ManagerID) = 3 (row 3 NULL ignored)
COUNT(Salary)    = 3 (row 3 NULL ignored)
COUNT(DISTINCT ManagerID) = 2 (101, 102)
```

Why This Design:

- COUNT(*) = Total records
- COUNT(column) = Non-NULL values in column
- Useful for data quality checks: `COUNT(*) - COUNT(column) = NULL count`

24. Can the **HAVING** clause be used without a **GROUP BY** clause, and how does SQL process such a query?

YES: HAVING can be used without GROUP BY (varies by database).

How It's Processed:

- Without GROUP BY, the entire result set is treated as one group
- HAVING filters that single group
- Result: Either 0 or 1 row

Example:

```
-- HAVING without GROUP BY (SQL Server, PostgreSQL)
SELECT COUNT(*) AS TotalEmployees, AVG(Salary) AS AvgSalary
FROM Employee
HAVING AVG(Salary) > 50000;

-- Result: 1 row if condition met, else 0 rows
```

Equivalent WITH GROUP BY:

```
SELECT COUNT(*) AS TotalEmployees, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY () -- Empty GROUP BY (treats all as one group)
HAVING AVG(Salary) > 50000;
```

Practical Use:

```
-- Check if average salary exceeds threshold
SELECT AVG(Salary) FROM Employee
HAVING AVG(Salary) > 50000;

-- More useful: Include aggregates with condition
SELECT COUNT(*) AS EmpCount, AVG(Salary) AS AvgSal
FROM Employee
HAVING COUNT(*) > 100 AND AVG(Salary) > 50000;
```

Database Compatibility:

- SQL Server, PostgreSQL: Support HAVING without GROUP BY
- MySQL: May require GROUP BY
- Oracle: May require GROUP BY

Window Functions

25. What is the purpose of window functions, and how are they different from aggregate functions?

Window Functions: Perform calculations on a "window" (set of rows) without collapsing result into single row.

Aggregate Functions: Collapse multiple rows into a single aggregated row.

Key Differences:

Aspect	Window Functions	Aggregate Functions
Result Rows	Same as input rows	One row per group
Details	Returns detail + aggregate	Only aggregate values
GROUP BY Required	No	Yes
Syntax	OVER clause	Standalone
Use Case	Ranking, running total, comparison	Summary statistics

Example:

```
-- Aggregate: 1 row total
SELECT Department, COUNT(*) AS EmpCount
FROM Employee
GROUP BY Department;
```

RESULT:

Department	EmpCount
IT	10
HR	5

```
-- Window: 15 rows (one per employee) + count
SELECT EmployeeID, Department,
       COUNT(*) OVER(PARTITION BY Department) AS DeptCount
FROM Employee;
```

RESULT:

EmployeeID	Department	DeptCount
1	IT	10
2	IT	10
...		
11	HR	5

Window Functions: ROW_NUMBER(), RANK(), LAG(), LEAD(), SUM() OVER(), etc.

26. What is the difference between ROW_NUMBER(), RANK(), and DENSE_RANK()?

ROW_NUMBER():

- Assigns unique numbers 1, 2, 3... to each row
- No gaps, doesn't skip numbers
- Treats tied values as different

RANK():

- Assigns same rank to tied values
- Skips numbers after tie
- Example: 1, 2, 2, 4 (skips 3)

DENSE_RANK():

- Assigns same rank to tied values
- No gaps in numbering
- Example: 1, 2, 2, 3 (no skip)

Example:

Salary: 100, 100, 90, 80

ROW_NUMBER()	= 1, 2, 3, 4	(always unique)
RANK()	= 1, 1, 3, 4	(skip after tie)
DENSE_RANK()	= 1, 1, 2, 3	(no skip)

When to Use:

- **ROW_NUMBER():** Pagination, when all rows must be unique
- **RANK():** Competition scoring, allow gaps
- **DENSE_RANK():** Ranking without gaps, level numbering

Syntax:

```
SELECT Name, Salary,  
       ROW_NUMBER() OVER(ORDER BY Salary DESC) AS RN,  
       RANK() OVER(ORDER BY Salary DESC) AS Rank,  
       DENSE_RANK() OVER(ORDER BY Salary DESC) AS DenseRank  
FROM Employee;
```

27. Why can't window functions be directly used in the WHERE clause?

Reason: SQL execution order processes WHERE before calculating window functions.

SQL Execution Order:

1. FROM
2. JOIN
3. WHERE (before window functions)
4. GROUP BY

5. HAVING
6. SELECT (window functions calculated here)
7. ORDER BY

Example of Error:

```
-- ERROR: Window function in WHERE
SELECT * FROM Employee
WHERE ROW_NUMBER() OVER(ORDER BY Salary) = 1;
-- Fails because WHERE executes before window functions
```

Correct Approach: Use Subquery or CTE:

```
-- Method 1: Subquery
SELECT * FROM (
    SELECT *,
        ROW_NUMBER() OVER(ORDER BY Salary DESC) AS RN
    FROM Employee
) t
WHERE RN = 1; -- Filter in outer query

-- Method 2: CTE
WITH RankedEmployees AS (
    SELECT *,
        ROW_NUMBER() OVER(ORDER BY Salary DESC) AS RN
    FROM Employee
)
SELECT * FROM RankedEmployees
WHERE RN = 1; -- Filter after CTE calculation
```

Window functions calculate in SELECT, filtering happens in WHERE → Incompatible timing.

28. What is the difference between **PARTITION BY** and **GROUP BY**?

PARTITION BY:

- Used in window functions (OVER clause)
- Divides rows into logical groups

- Each row retains its identity
- Doesn't reduce result set
- Calculates aggregate per partition for each row

GROUP BY:

- Aggregates rows into groups
- Collapses rows within each group
- Returns one row per group
- Reduces result set
- Requires aggregate functions

Example:

Employee Data:

ID	Name	Department	Salary
1	John	IT	50000
2	Jane	IT	60000
3	Bob	HR	55000
4	Alice	HR	65000

-- GROUP BY (3 rows: 1 IT group, 1 HR group, 1 total)

```
SELECT Department, COUNT(*), AVG(Salary)
FROM Employee
GROUP BY Department;
```

RESULT:

Department	COUNT	AVG
IT	2	55000
HR	2	60000

-- PARTITION BY (4 rows: detail + partition aggregate)

```
SELECT ID, Name, Department, Salary,
       AVG(Salary) OVER(PARTITION BY Department) AS DeptAvgSalary
FROM Employee;
```

RESULT:

ID	Name	Department	Salary	DeptAvgSalary
1	John	IT	50000	55000
2	Jane	IT	60000	55000
3	Bob	HR	55000	60000
4	Alice	HR	65000	60000

Key Difference: PARTITION BY preserves detail rows; GROUP BY aggregates them.

29. In what scenarios are `LAG()` and `LEAD()` functions commonly used?

LAG(): Accesses previous row's value in the result set. **LEAD():** Accesses next row's value in the result set.

Common Scenarios:

1. Year-over-Year Comparison:

```
SELECT Year, Revenue,  
       LAG(Revenue) OVER(ORDER BY Year) AS PreviousYearRevenue,  
       Revenue - LAG(Revenue) OVER(ORDER BY Year) AS Growth  
FROM AnnualSales;
```

2. Change Detection:

```
SELECT Date, Price,  
       LAG(Price) OVER(ORDER BY Date) AS PreviousPrice,  
       CASE WHEN Price > LAG(Price) OVER(ORDER BY Date)  
            THEN 'Increase' ELSE 'Decrease' END AS PriceChange  
FROM StockPrice;
```

3. Gap Detection:

```
SELECT Date,  
       LEAD(Date) OVER(ORDER BY Date) - Date AS DaysUntilNext  
FROM OrderLog;
```

4. Running Differences:

```
SELECT Quarter, Sales,  
       LEAD(Sales) OVER(ORDER BY Quarter) AS NextSales,  
       LEAD(Sales) OVER(ORDER BY Quarter) - Sales AS Difference  
FROM QuarterlySales;
```

Syntax:

```
LAG(column, offset, default) OVER(ORDER BY column)
LEAD(column, offset, default) OVER(ORDER BY column)

-- Example: Get previous 2 values or NULL if unavailable
LAG(Salary, 2) OVER(ORDER BY Date)
```

30. How do window functions improve query readability compared to correlated subqueries?

Window Functions:

- Clean, explicit syntax
- Easy to understand intent
- Better performance (optimized by query engine)
- Maintainable code

Correlated Subqueries:

- Complex nested logic
- Harder to follow
- Often slower (subquery runs per row)
- Difficult to maintain

Side-by-Side Comparison:

Find highest salary per department using Correlated Subquery:

```
SELECT Name, Department, Salary
FROM Employee e1
WHERE Salary = (
    SELECT MAX(Salary)
    FROM Employee e2
    WHERE e2.Department = e1.Department
);
-- Subquery executes for each row (poor performance)
```

Find highest salary per department using Window Function:

```
SELECT * FROM (  
    SELECT Name, Department, Salary,  
           MAX(Salary) OVER(PARTITION BY Department) AS MaxSalary  
    FROM Employee  
    ) t  
WHERE Salary = MaxSalary;  
-- Window function executes once (better performance)
```

Readability Comparison:

Aspect	Correlated Subquery	Window Function
Readability	Hard to follow nesting	Clear and explicit
Performance	Slower (per-row subquery)	Faster (single pass)
Complexity	Multiple query levels	Single level
Maintenance	Difficult	Easy
SQL Version	Available in older SQL	Modern SQL standard

Recommendation: Use window functions for better performance and readability.

EXISTS, IN and Subqueries

31. What is the difference between **IN** and **EXISTS**, and how does performance differ for large datasets?

IN Subquery:

- Checks if value EXISTS in a list
- Builds complete list in memory first
- Returns TRUE/FALSE for each value
- Processes all subquery results

EXISTS Subquery:

- Checks IF matching rows exist
- Stops after finding first match
- Returns TRUE/FALSE (no list)
- More efficient (semi-join)

Syntax Comparison:

```
-- IN: Builds list of all matching values
SELECT * FROM Customers
WHERE CustomerID IN (
    SELECT CustomerID FROM Orders
);

-- EXISTS: Checks existence only
SELECT * FROM Customers c
WHERE EXISTS (
    SELECT 1 FROM Orders o
    WHERE o.CustomerID = c.CustomerID
);
```

Performance Difference - Large Datasets:

Factor	IN	EXISTS
Subquery Processing	All rows	Stops at first match
Memory Usage	Higher (full list)	Lower (first match only)
Speed	Slower for large datasets	Faster
NULL Handling	Can cause issues with NOT IN	Cleaner
Index Usage	Limited	Better optimizer support

Rule of Thumb:

- Small subqueries (< 1000 rows): IN is fine
- Large subqueries (> 1000 rows): EXISTS is better
- Correlated query: EXISTS preferred
- Simple list: IN acceptable

Best Practice: Prefer EXISTS for large subqueries and production code.

32. Why does **NOT IN** sometimes return unexpected results when **NULL** values are present?

The Problem with NOT IN and NULL: NOT IN doesn't work as expected when the subquery contains NULL values.

Example:

```
Orders Table:
CustomerID: 1, 2, 5, NULL

-- Unexpected Result: Returns 0 rows!
SELECT * FROM Customers
WHERE CustomerID NOT IN (
    SELECT CustomerID FROM Orders
);

-- Why? NULL breaks NOT IN logic:
IF CustomerID NOT IN (1, 2, 5, NULL):
    NOT (CustomerID = 1 OR CustomerID = 2 OR CustomerID = 5 OR CustomerID = NULL)

Since NULL = anything returns UNKNOWN:
NOT (FALSE OR FALSE OR FALSE OR UNKNOWN) = NOT UNKNOWN = UNKNOWN

UNKNOWN is treated as FALSE in WHERE → No rows returned
```

Solution 1: Exclude NULL from Subquery

```
SELECT * FROM Customers
WHERE CustomerID NOT IN (
    SELECT CustomerID FROM Orders
    WHERE CustomerID IS NOT NULL -- Exclude NULLs
);
```

Solution 2: Use NOT EXISTS

```
SELECT * FROM Customers c
WHERE NOT EXISTS (
    SELECT 1 FROM Orders o
    WHERE o.CustomerID = c.CustomerID
);
-- NOT EXISTS handles NULL correctly
```

Key Takeaway: NOT IN + NULL = Unexpected results. Use NOT EXISTS or exclude NULLs.

33. What is the difference between a correlated subquery and a non-correlated subquery?

Non-Correlated Subquery (Independent):

- Executes ONCE
- Independent of outer query
- Inner query doesn't reference outer table
- Faster execution
- Result used as constant/list

Correlated Subquery (Dependent):

- Executes once per OUTER QUERY ROW
- References outer query columns
- Inner query depends on outer row values
- Slower execution (row-by-row)
- Like a loop in programming

Example Comparison:

```
-- NON-CORRELATED (executes once, returns list [1,2,5]):
SELECT * FROM Customers
WHERE CustomerID IN (
    SELECT CustomerID FROM Orders -- Doesn't reference outer table
);
-- Subquery: SELECT CustomerID FROM Orders → [1, 2, 5]
-- Then: SELECT * FROM Customers WHERE CustomerID IN (1, 2, 5)

-- CORRELATED (executes for each customer row):
SELECT * FROM Customers c
WHERE EXISTS (
    SELECT 1 FROM Orders o
    WHERE o.CustomerID = c.CustomerID -- References outer table c
);
-- For each customer:
-- SELECT 1 FROM Orders WHERE CustomerID = 1
-- SELECT 1 FROM Orders WHERE CustomerID = 2
-- ...
```

Performance Comparison:

Aspect	Non-Correlated	Correlated
Execution Count	1	Once per outer row
Speed	Fast	Slow (100 rows = 100 executions)
Optimization	Better	Limited
Use Case	Finding lists	Row-by-row comparison
Example	IN subquery	EXISTS, comparison functions

Performance Example:

- Non-correlated: 1 subquery + 1 main query = 2 executions
- Correlated (100 rows): 1 + 100 = 101 executions

Best Practice: Avoid correlated subqueries when possible; use JOINS or window functions.

34. What are the advantages and disadvantages of using Common Table Expressions (CTEs) instead of subqueries?

CTEs (Common Table Expressions):

- Named temporary result set
- Defined using WITH clause
- Readable, reusable within query
- Can be recursive

Advantages of CTEs:

1. **Readability:** Top-to-bottom logic flow
2. **Reusability:** Reference same CTE multiple times
3. **Maintainability:** Named, self-documenting
4. **Recursion:** Support hierarchical queries
5. **Debugging:** Easier to test individual CTEs

Disadvantages of CTEs:

1. **Performance:** May not optimize as well as subqueries
2. **Not Materialized:** CTE results not stored (depends on DB)
3. **Scope:** CTE scope limited to single query
4. **Complexity:** Multiple CTEs can get complex
5. **Nesting Limit:** Very deep CTEs harder to manage

Syntax Comparison:

```
-- SUBQUERY (nested, harder to read):
SELECT * FROM (
    SELECT * FROM (
        SELECT * FROM Orders WHERE Amount > 1000
    ) o
    WHERE o.OrderDate > '2023-01-01'
) recent_orders;

-- CTE (cleaner, top-to-bottom):
WITH large_orders AS (
    SELECT * FROM Orders WHERE Amount > 1000
),
recent_orders AS (
    SELECT * FROM large_orders WHERE OrderDate > '2023-01-01'
)
SELECT * FROM recent_orders;
```

When to Use CTE:

- Complex queries with multiple subqueries
- Recursive queries (hierarchies)
- Reusing same subquery multiple times
- Improved readability needed

When to Use Subquery:

- Simple queries
- Single-use results
- Performance critical (some DBs optimize better)

35. What is a recursive Common Table Expression (CTE), and where is it commonly used?

Recursive CTE: A CTE that references itself to process hierarchical data.

Structure:

```

WITH RECURSIVE cte_name AS (
    -- ANCHOR: Base case (starting point)
    SELECT columns FROM table WHERE condition

    UNION ALL

    -- RECURSIVE: Self-reference for traversal
    SELECT columns FROM table
    JOIN cte_name ON join_condition
    WHERE termination_condition
)
SELECT * FROM cte_name;

```

Common Use Cases:

1. Organizational Hierarchy:

```

WITH RECURSIVE EmployeeHierarchy AS (
    -- Base: Top-level employees (no manager)
    SELECT EmployeeID, Name, ManagerID, 1 AS Level
    FROM Employee
    WHERE ManagerID IS NULL

    UNION ALL

    -- Recursive: Direct reports of found employees
    SELECT e.EmployeeID, e.Name, e.ManagerID, h.Level + 1
    FROM Employee e
    JOIN EmployeeHierarchy h ON e.ManagerID = h.EmployeeID
)
SELECT * FROM EmployeeHierarchy;

```

2. Category Tree:

```

WITH RECURSIVE CategoryTree AS (
    SELECT CategoryID, CategoryName, ParentID, 1 AS Level
    FROM Category WHERE ParentID IS NULL

    UNION ALL

    SELECT c.CategoryID, c.CategoryName, c.ParentID, ct.Level + 1
    FROM Category c
    JOIN CategoryTree ct ON c.ParentID = ct.CategoryID
)
SELECT * FROM CategoryTree;

```

3. Bill of Materials:

- Product component hierarchies
- Assembly structures
- Part breakdowns

4. Number Series:

```
WITH RECURSIVE Numbers AS (  
    SELECT 1 AS n  
    UNION ALL  
    SELECT n + 1 FROM Numbers WHERE n < 100  
)  
SELECT * FROM Numbers;
```

Key Points:

- Anchor query starts recursion
- UNION ALL combines results
- Recursive part joins with CTE itself
- Must have termination condition to prevent infinite loop

36. What is the difference between a CTE, a temporary table, and a derived table?

CTE (Common Table Expression):

- Temporary named result set
- Defined with WITH clause
- Scope: Current query only
- Created on-the-fly, no storage
- Reusable within same query

Temporary Table:

- Physical table created in tempdb/temp schema

- Exists for session duration (session temp) or connection (global temp)
- Can be indexed, have statistics
- Persists across queries in same session
- Can be dropped explicitly

Derived Table (Subquery in FROM):

- Inline subquery in FROM clause
- Must be aliased
- No name outside scope
- Exists only for current query
- Cannot reference other derived tables at same level

Comparison Table:

Aspect	CTE	Temp Table	Derived Table
Definition	WITH clause	CREATE TABLE	SELECT in FROM
Scope	Current query	Session/Connection	Current query
Reusable	Yes, in same query	Yes, across queries	No, single use
Indexing	No	Yes	No
Storage	Memory	Disk (tempdb)	Memory
Performance	Good	Best (if indexed)	Good
Naming	Named, self-documented	Named, persistent	Required alias only

Examples:

```
-- CTE:
WITH ranked_employees AS (
    SELECT *, ROW_NUMBER() OVER(ORDER BY Salary DESC) AS Rank
    FROM Employee
)
SELECT * FROM ranked_employees WHERE Rank <= 10;

-- Temporary Table:
CREATE TEMPORARY TABLE temp_ranked_employees AS
SELECT *, ROW_NUMBER() OVER(ORDER BY Salary DESC) AS Rank
FROM Employee;
SELECT * FROM temp_ranked_employees WHERE Rank <= 10;
DROP TABLE temp_ranked_employees;

-- Derived Table:
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER(ORDER BY Salary DESC) AS Rank
    FROM Employee
) ranked_employees
WHERE Rank <= 10;
```

When to Use:

- **CTE:** Moderate complexity, better readability, single query
- **Temp Table:** Reuse across multiple queries, need indexing, large dataset
- **Derived Table:** Simple inline transformations, one-time use

Indexes and Performance

37. What is an index, and how does it improve query performance?

Index: A database structure that maps column values to row locations (like book index).

How It Works:

- Creates sorted structure (usually B-tree)
- Enables quick lookup without scanning all rows
- Similar to dictionary: Look up word → Page number

Performance Improvement:

- **Without Index:** Full table scan, checks every row $O(n)$
- **With Index:** Binary search, finds row quickly $O(\log n)$
- Example: 1 million rows, index reduces from ~1M to ~20 operations

Types of Indexes:

1. **Clustered Index:** Table data sorted in index order (one per table)
2. **Non-Clustered Index:** Separate index, points to data (many allowed)
3. **Unique Index:** Enforces uniqueness
4. **Composite Index:** Multiple columns
5. **Full-Text Index:** Text search optimization

Example:

```
-- Without index: SELECT * FROM Employee WHERE EmployeeID = 123;
-- Scans all 1,000,000 rows

-- Create index:
CREATE INDEX idx_employee_id ON Employee(EmployeeID);

-- With index: SELECT * FROM Employee WHERE EmployeeID = 123;
-- Finds row in ~20 operations
```

Trade-off:

- **Benefit:** Fast SELECT queries
- **Cost:** Slower INSERT/UPDATE/DELETE (must update index)

38. What is the difference between clustered and non-clustered indexes?

Clustered Index:

- Defines the PHYSICAL ORDER of table data
- One per table only
- Table data stored sorted by clustered index key

- Leaf nodes contain actual data pages
- Usually on primary key
- Faster if searching on indexed column

Non-Clustered Index:

- Separate index structure (doesn't affect data order)
- Up to 999 per table (varies by DB)
- Leaf nodes contain index key + row locator (bookmark)
- Points to clustered index or heap
- Can be created on any column(s)
- Useful for covering queries

Visual Comparison:

Clustered Index (Primary Key - ID):

Index	Data Page
1	[1, John, 50000]
2	[2, Jane, 60000]
3	[3, Bob, 55000]

----- Physical table data is sorted -----

Non-Clustered Index (Salary):

Index	Clustered Key
50000	1
55000	3
60000	2

----- Points back to clustered index -----

Comparison:

Aspect	Clustered	Non-Clustered
Per Table	1 only	999+
Data Order	Changes table order	No change
Leaf Nodes	Actual data	Index + bookmark
Speed	Fastest for clustered column	Faster than full scan
Space	Same as data	Extra storage
Seek/Scan	Fast	Fast (if used)

Example:

```
CREATE CLUSTERED INDEX idx_emp_id ON Employee(EmployeeID);  
CREATE NONCLUSTERED INDEX idx_emp_name ON Employee(Name);
```

39. Why can indexes improve **SELECT** queries but slow down **INSERT**, **UPDATE**, and **DELETE** operations?

Why Indexes Help SELECT:

- Allow quick row lookups (no full scan)
- Enable early filtering
- Support efficient sorting (pre-sorted)
- Accelerate JOINS (foreign key indexes)

Why Indexes Slow Down DML (INSERT/UPDATE/DELETE):**INSERT:**

- Must insert data into table
- Must also insert into every index
- Maintains sorted index structure (requires reordering)
- Multiple index updates = slower

UPDATE:

- If indexed column changes, index must be updated
- Remove old entry, add new entry
- Restructure index (B-tree rebalancing)
- Multiple columns = multiple index updates

DELETE:

- Remove from table
- Remove from all indexes
- Rebalance index structures

- More indexes = more work

Example Performance Impact:

```
-- Insert with 0 indexes: 1 ms
INSERT INTO Employee VALUES (...);

-- Insert with 5 indexes: 50 ms
-- Must update index for each column in each index

-- Query without index: 1000 ms (full scan)
SELECT * FROM Employee WHERE Name = 'John';

-- Query with index: 1 ms (index seek)
```

Index Maintenance Costs:

Operation	Time	Reason
INSERT 1 row no index	1 ms	Single table insert
INSERT 1 row + 1 index	5-10 ms	Maintain 1 index
INSERT 1 row + 5 indexes	50 ms	Maintain 5 indexes

Trade-off Decision:

- READ-heavy workloads: Add many indexes
- WRITE-heavy workloads: Use fewer indexes
- BALANCED: Index only frequently searched columns

40. Why should indexes not be created on every column in a table?

Costs of Over-Indexing:

1. Storage Space:

- Each index consumes disk space
- 100 columns with indexes = 100x data size
- Extra space needed for backup/replication

2. DML Performance:

- INSERT/UPDATE/DELETE slower (maintain all indexes)
- 10 indexes = ~10x slower writes
- Locks and contention increase

3. Memory Usage:

- Indexes loaded into memory
- Less memory for data cache
- Query performance may degrade

4. Maintenance Overhead:

- Statistics updates on all indexes
- Index fragmentation (reorganize, rebuild)
- DBMS optimizer complexity

5. Query Optimizer Confusion:

- Too many index choices slows optimization
- Suboptimal index selection
- Query plans get complex

When to Index a Column:

- ✓ Frequently used in WHERE clause
- ✓ Used in JOINS or ORDER BY
- ✓ Searched frequently (high selectivity)
- ✓ Used in aggregate functions
- ✓ Not too many duplicate values
- X Rarely searched or updated frequently
- X Low selectivity (many duplicates)
- X Boolean columns (only 2 values)
- X Updated very frequently
- X Never used in queries

Best Practice:

Index only columns that:

1. Are frequently searched
2. Have high selectivity (few duplicates)
3. Improve query performance measurably

Typical ratio: 1-3 indexes per table, not per column

Example:

```
-- DON'T DO THIS:
CREATE INDEX idx_id ON Employee(EmployeeID);
CREATE INDEX idx_name ON Employee(Name);
CREATE INDEX idx_age ON Employee(Age);
CREATE INDEX idx_gender ON Employee(Gender); -- Only 2 values!
CREATE INDEX idx_city ON Employee(City);

-- DO THIS:
CREATE INDEX idx_emp_id ON Employee(EmployeeID); -- Primary key
CREATE INDEX idx_emp_name ON Employee(Name); -- Frequently searched
CREATE INDEX idx_emp_dept ON Employee(DepartmentID); -- Foreign key
```

41. What is a composite index, and why does column order matter in a composite index?

Composite Index: Index on multiple columns together.

Syntax:

```
CREATE INDEX idx_emp_dept_salary ON Employee(Department, Salary);
```

Why Column Order Matters:

Composite indexes use **leftmost prefix principle** — queries can only use starting columns.

Example:

Index: (Department, Salary, Name)

- ✓ Can use: Department
- ✓ Can use: Department + Salary
- ✓ Can use: Department + Salary + Name
- ✗ Cannot use efficiently: Salary alone
- ✗ Cannot use efficiently: Salary + Name
- ✗ Cannot use efficiently: Name alone

Real Example:

```
-- Index: (Department, Salary)
CREATE INDEX idx_dept_sal ON Employee(Department, Salary);

-- EFFICIENT (uses index):
SELECT * FROM Employee WHERE Department = 'IT';
SELECT * FROM Employee WHERE Department = 'IT' AND Salary > 50000;

-- INEFFICIENT (index not fully used):
SELECT * FROM Employee WHERE Salary > 50000; -- Starts at Salary, not leftmost

-- INEFFICIENT (full scan):
SELECT * FROM Employee WHERE Name = 'John'; -- Name is rightmost
```

Ordering Strategy:

1. **Equality Conditions First:** Columns in WHERE with = (filters most)
2. **Range Conditions Next:** Columns with >, <, BETWEEN (filters after)
3. **Sorting/Filtering Last:** Columns in ORDER BY (lowest cardinality)

Example Optimization:

```
-- Query patterns:
WHERE Department = 'IT' AND Salary > 50000 ORDER BY Name

-- Best index order:
CREATE INDEX idx ON Employee(Department, Salary, Name);
-- Department: Equality (filters first)
-- Salary: Range (narrows further)
-- Name: Sorting (final ordering)
```

Selectivity Consideration:

```
-- If Gender (M/F) and Department (many distinct):  
-- GOOD: (Department, Gender) -- More selective first  
-- BAD: (Gender, Department) -- Less selective first
```

42. What is index selectivity, and why is it important for query optimization?

Index Selectivity: Measure of how many rows an index filters out (how "selective" it is).

Calculation:

$\text{Selectivity} = (\text{Distinct Values}) / (\text{Total Rows}) * 100\%$

Higher selectivity = Better filtering = Better index

Examples:

-- EmployeeID: 1000 distinct values, 1000 rows

Selectivity = $1000/1000 = 100\%$ (EXCELLENT)

-- Department: 10 distinct values, 1000 rows

Selectivity = $10/1000 = 1\%$ (POOR)

-- Gender: 2 distinct values, 1000 rows

Selectivity = $2/1000 = 0.2\%$ (TERRIBLE)

Why It Matters:

High Selectivity (Good for Index):

- Index eliminates most rows
- Query returns few rows
- Index seek is efficient
- Example: EmployeeID, Email, SSN

Low Selectivity (Bad for Index):

- Index eliminates few rows
- Query returns many rows
- Full scan might be faster
- Example: Gender, Status (Active/Inactive)

Decision Table:

Selectivity	Efficiency	Recommendation
> 95%	Excellent	Always index
90-95%	Good	Index
50-90%	Fair	Conditional
10-50%	Poor	Avoid
< 10%	Very Poor	Full scan better

Real Query Example:

```
-- Bad Selectivity: Returns 500 of 1000 rows
SELECT * FROM Employee WHERE Status = 'Active';
-- Index might not be used (full scan faster)

-- Good Selectivity: Returns 1 of 1000 rows
SELECT * FROM Employee WHERE EmployeeID = 123;
-- Index definitely used (quick seek)
```

Optimization Tips:

1. Index high-selectivity columns first
2. Check selectivity before creating index: `SELECT COUNT(DISTINCT column)/COUNT(*) FROM table;`
3. Avoid indexing low-selectivity columns
4. Monitor query performance after index creation

43. Why does the database optimizer sometimes ignore an available index and perform a full table scan instead?

Reasons Optimizer Ignores Index:

1. **Low Selectivity:** Index would return too many rows (>5-10%), full scan faster
2. **Index Fragmentation:** Heavily fragmented index (>30%), sequential scan faster
3. **Missing Statistics:** Optimizer doesn't know row distribution correctly
4. **Cost-Based Decision:** Estimates index cost higher than scan cost
5. **Data Type Mismatch:** Type conversion makes index unusable
6. **Random vs Sequential I/O:** Multiple seeks slower than one scan

Example:

Index on Status returns 500 of 1000 rows:
- Index route: 500 seeks + 500 lookups = 1000ms
Table scan: 1 scan = 50ms
Optimizer chooses SCAN

Prevention: Update statistics, defragment index, improve selectivity, use covering index.

Transactions and Concurrency

44. What are the ACID properties of a transaction, and why are they important in database systems?

ACID Properties: Guarantee reliable database transactions.

A - Atomicity (All or Nothing):

- Transaction either completes fully or doesn't execute at all
- No partial updates
- If error occurs, automatic ROLLBACK
- Example: Money transfer – debit AND credit, not just one

C - Consistency (Valid State):

- Database moves from one valid state to another
- Constraints, rules, business logic maintained
- Referential integrity preserved
- Example: Total money in accounts stays same after transfer

I - Isolation (No Interference):

- Concurrent transactions don't interfere with each other
- One transaction's intermediate state invisible to others
- Dirty reads, phantom reads prevented
- Level: Read Uncommitted, Read Committed, Repeatable Read, Serializable

D - Durability (Permanent):

- Committed data survives system failures
- Stored in persistent storage (disk)
- No loss even if power fails
- Backup/Recovery ensures recoverability

Example - Bank Transfer \$100:

```
BEGIN TRANSACTION
  UPDATE Account SET Balance = Balance - 100 WHERE ID = 1; -- Atomicity: Both
or neither
  UPDATE Account SET Balance = Balance + 100 WHERE ID = 2; -- Consistency:
Total preserved
COMMIT; -- Durability: Persisted to disk
        -- Isolation: Other users don't see intermediate state
```

Why Important:

- Data integrity: Reliable, predictable state
- Business compliance: Regulations require transaction safety
- Multi-user safety: Concurrent operations don't corrupt data
- Failure recovery: System can recover from crashes

45. What is the difference between **COMMIT** and **ROLLBACK** in SQL transactions?

COMMIT:

- Finalizes transaction, makes changes permanent
- Writes changes to persistent storage
- Changes visible to other users/sessions
- Cannot be undone (unless logged/backed up)
- Releases locks held during transaction

ROLLBACK:

- Cancels transaction, undoes all changes
- Reverts to state before transaction started
- Changes are NOT applied to database
- Useful for error handling
- Also releases locks

Example:

```
BEGIN TRANSACTION
UPDATE Employee SET Salary = 60000 WHERE ID = 1;

IF @@ERROR <> 0 -- Check for errors
    ROLLBACK;   -- Undo changes
ELSE
    COMMIT;     -- Make permanent
END
```

Comparison:

Aspect	COMMIT	ROLLBACK
Effect	Makes permanent	Undoes all
Visibility	Visible to all users	Not visible to others
Reversibility	Cannot undo	Discards changes
Use Case	Success path	Error handling
Locks	Released	Released

When to Use:

- **COMMIT:** After successful operations
- **ROLLBACK:** On error, validation failure, or business logic issue

Auto-Commit Setting:

- In many databases, auto-commit is ON by default
- Each statement auto-commits if no error
- Disable: `SET AUTOCOMMIT OFF;` (MySQL) or `BEGIN TRANSACTION;` (SQL Server)

46. What are transaction isolation levels, and how do they affect data consistency and concurrency?

Transaction Isolation Levels: Define how much concurrent transaction interference is allowed.

Four Levels (from loosest to strictest):

1. Read Uncommitted (Isolation Level 0):

- Lowest isolation, highest concurrency
- Can read uncommitted changes from other transactions
- Problems: Dirty reads, non-repeatable reads, phantom reads
- Speed: Fastest
- Use Case: Rare, only for read-only non-critical data

2. Read Committed (Isolation Level 1):

- Default in most databases
- Can only read committed changes
- Problems: Non-repeatable reads, phantom reads
- Speed: Good
- Use Case: Most applications

3. Repeatable Read (Isolation Level 2):

- Cannot modify or delete rows read by transaction
- Problems: Phantom reads possible
- Speed: Slower
- Use Case: Need consistent view of same data

4. Serializable (Isolation Level 3):

- Highest isolation, lowest concurrency
- Transactions run as if sequential (no overlap)
- No concurrency issues
- Speed: Slowest
- Use Case: Critical operations where consistency critical

Comparison Table:



Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Yes	Yes	Yes
Read Committed	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No

Example:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN TRANSACTION
  SELECT * FROM Account WHERE Balance > 1000;
  -- No other transaction can modify these rows until commit
COMMIT;
```

Trade-off: Stricter isolation = Better consistency but lower concurrency.

47. What are dirty reads, non-repeatable reads, and phantom reads, and how are they prevented?

Dirty Reads:

- Reading uncommitted changes from another transaction
- Transaction A modifies row, Transaction B reads it BEFORE commit
- If A rolls back, B read invalid "dirty" data

Prevention: Read Committed or higher isolation level

Example:

Transaction A:	Transaction B:
<code>BEGIN TRANSACTION</code>	<code>SET ISOLATION LEVEL READ UNCOMMITTED</code>
<code>UPDATE Account SET Balance = 0</code>	<code>SELECT Balance FROM Account -- Sees 0</code>
<code>(dirty)</code>	
<code>ROLLBACK -- Reverts!</code>	

Non-Repeatable Reads:

- Reading same row twice in transaction returns different values
- Another transaction modifies/deletes the row between reads
- Loss of data consistency within single transaction

Prevention: Repeatable Read or higher isolation level

Example:

Transaction A:	Transaction B:
<code>BEGIN</code>	
<code>SELECT Balance FROM Account</code> (reads 100)	<code>UPDATE Account SET Balance = 200</code>
<code>SELECT Balance FROM Account</code> (reads 200 -- Different!)	
<code>COMMIT</code>	

Phantom Reads:

- Query returns different set of rows each time
- Another transaction inserts/deletes rows matching WHERE clause
- Rows appear or disappear between queries

Prevention: Serializable isolation level**Example:**

Transaction A:	Transaction B:
<code>SELECT COUNT(*) FROM Account</code> WHERE Balance > 1000 (reads 5)	<code>INSERT INTO Account VALUES (...)</code> -- Adds account
<code>SELECT COUNT(*) FROM Account</code> WHERE Balance > 1000 (reads 6 -- Phantom!)	
<code>COMMIT</code>	

Prevention Summary:

Problem	Cause	Prevention
Dirty Read	Reads uncommitted	Read Committed+
Non-Repeatable Read	Row modified between reads	Repeatable Read+
Phantom Read	Rows added/deleted in range	Serializable

Locking Mechanism:

- Shared locks: Prevent writes

- Exclusive locks: Prevent reads and writes
- Range locks: Prevent inserts in range (phantom prevention)

48. What is a deadlock, and how does a database detect and resolve it?

Deadlock: Two or more transactions waiting infinitely for each other to release locks.

Deadlock Example:

Transaction A: Lock Resource 1 Wait for Resource 2 ↓ Waiting forever...	Transaction B: Lock Resource 2 Wait for Resource 1 ↓ Waiting forever...
---	---

Typical Scenario:

Transaction A: <code>UPDATE Account SET Balance = 0 WHERE ID = 1 (locks row 1)</code> <code>UPDATE Account SET Balance = 0 WHERE ID = 2 (waits for B to unlock row 2)</code>	Transaction B: <code>UPDATE Account SET Balance = 0 WHERE ID = 2 (locks row 2)</code> <code>UPDATE Account SET Balance = 0 WHERE ID = 1 (waits for A to unlock row 1)</code>
--	--

DEADLOCK! Both waiting infinitely.

How Database Detects Deadlock:

1. **Wait-for Graph:** Tracks transactions waiting for locks
2. **Cycle Detection:** Checks if circular wait exists
3. **Timeout:** If transaction waits too long (e.g., 5 seconds)
4. **Lock Manager:** Continuously monitors lock waits

How Database Resolves Deadlock:

1. **Victim Selection:**

- Chooses least costly transaction to rollback
- Considers: Transaction cost, work done, priority
- Usually youngest transaction chosen (least work)

2. Rollback Victim:

- Victim transaction is rolled back
- Locks released
- Other transaction proceeds
- Victim can retry

3. Error Reporting:

- Victim gets error (e.g., "deadlock detected")
- Application handles with retry logic

Deadlock Prevention Strategies:

1. Consistent Lock Order:

```
-- ALWAYS lock in same order
Transaction A:      Transaction B:
UPDATE Account(1)   UPDATE Account(1)  -- Same order!
UPDATE Account(2)   UPDATE Account(2)
```

2. Minimize Lock Duration:

```
BEGIN TRANSACTION
  -- Do work here (short)
COMMIT;  -- Release locks quickly
```

3. Use Lower Isolation Level:

- Lower isolation = Fewer locks = Less deadlock risk

4. Avoid Hot Spots:

- Don't update same popular rows frequently
- Partition writes across different rows

Example Retry Logic:

```
Retry counter = 0
WHILE Retry counter < 3:
    BEGIN TRANSACTION
    -- Perform operations
    COMMIT
EXCEPT DEADLOCK
    ROLLBACK
    Retry counter++
    WAIT 100ms
END WHILE
```

Advanced SQL

49. Why can two SQL queries that return the same result have significantly different execution times?

Reasons for Different Execution Times:

1. Query Optimization Difference:

- Different execution plans generated
- Join order matters: A join B vs B join A
- One plan uses indexes, other uses full scan
- Subquery vs JOIN: Different optimization strategies

Example:

```
-- Query 1 (slow): Full scan then filter
SELECT * FROM Orders o
WHERE EXISTS (
    SELECT 1 FROM Customers c
    WHERE c.CustomerID = o.CustomerID
);

-- Query 2 (fast): Index seek with semi-join
SELECT DISTINCT o.* FROM Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID;
```

2. Index Usage:

- Query 1: No index use → full table scan
- Query 2: Uses index → quick seek

3. Data Distribution:

- Same queries on different data sets perform differently
- 10 rows vs 1 million rows: Very different times
- Index effectiveness changes with data

4. Query Complexity:

- Correlated vs Non-correlated subqueries
- Correlated: Executes per row (N executions)
- Non-correlated: Executes once

Factors Affecting Execution Time:

- **Cardinality:** Number of rows returned
- **Table Size:** Larger tables = more I/O
- **Index Selectivity:** How well index filters
- **CPU vs I/O:** Computation vs disk access
- **Memory:** Available for caching
- **Lock Contention:** Waiting for other transactions

Performance Optimization:

1. Use EXPLAIN/execution plan to analyze
2. Check indexes are being used

3. Avoid correlated subqueries
4. Use JOINS over subqueries
5. Update statistics
6. Consider index hints if necessary

50. What are the most common reasons for poor SQL query performance, and how would you identify and resolve them?

Common Performance Issues:

1. Missing or Poor Indexes:

- Full table scans on large tables
- Slow WHERE, JOIN, ORDER BY conditions
- Resolution: Add appropriate indexes, check selectivity

2. Index Fragmentation:

- Heavy fragmentation (>30%) causes slow seeks
- Resolution: REBUILD or REORGANIZE index

3. Outdated Statistics:

- Optimizer makes wrong cardinality estimates
- Chooses suboptimal execution plan
- Resolution: `UPDATE STATISTICS` or `ANALYZE TABLE`

4. Correlated Subqueries:

- Executes once per row (N+1 queries)
- Very slow for large datasets
- Resolution: Convert to JOINS or window functions

5. SELECT * (Unnecessary Columns):

- Retrieves all columns, wastes I/O
- Transfers unnecessary data

- Resolution: Select only needed columns

6. Large Result Sets:

- Returns millions of rows
- Wastes network/memory
- Resolution: Add WHERE filters, pagination (LIMIT/OFFSET)

7. JOIN Order:

- Bad join sequence multiplies row counts
- Example: Join 1M rows with 1M rows
- Resolution: Filter largest tables first

8. Data Type Mismatch:

- `WHERE EmployeeID = '123'` (string vs integer)
- Forces index scan instead of seek
- Resolution: Use correct data types in comparison

How to Identify Performance Issues:

1. Execution Plan Analysis:

```
EXPLAIN PLAN FOR
SELECT * FROM Employee WHERE Salary > 50000;

-- Look for: Full table scans, high costs, many rows
```

2. Query Statistics:

- Execution time
- Rows returned
- CPU time
- I/O statistics

3. Slow Query Logs:

- Queries running > threshold time
- Enable: `SET GLOBAL slow_query_log = ON;`

4. Monitoring Tools:

- SQL Server: Management Studio, Azure Data Studio
- MySQL: MySQL Workbench, Percona Monitoring
- PostgreSQL: PgAdmin, pg_stat_statements

Resolution Approach:

1. Identify slow query (execution plan, logs)
2. Analyze: Missing index, bad join, data type mismatch
3. Test: Create index, rewrite query, update stats
4. Verify: Check execution time improvement
5. Monitor: Ensure fix doesn't break other queries

Quick Checklist:

- Execution plan shows index seeks (not scans)
- Statistics are up-to-date
- Indexes not fragmented
- No correlated subqueries
- WHERE/JOIN conditions use correct data types
- No SELECT * on large tables
- Appropriate indexes exist

Bonus: Frequently Asked "Why" Questions

* Why is **SELECT *** considered a bad practice in production environments?

**Problems with SELECT :*

1. Unnecessary Data Transfer:

- Retrieves all columns even if only few needed
- Wastes network bandwidth
- Slower over slow connections

2. Memory Usage:

- Loads extra data into memory
- Depletes cache available for other queries
- More expensive in cloud (billed by data transferred)

3. Application Performance:

- Application processes unused columns
- Slower serialization/deserialization
- More memory consumption on client side

4. Index Effectiveness Lost:

- Cannot use covering indexes (index contains all columns)
- Index must still access table data
- Negates index optimization benefits

5. Schema Change Risk:

- New column added to table
- `SELECT *` suddenly includes it (unexpected)
- May break application logic
- Hidden dependency on table structure

6. Query Plan Issues:

- Optimizer may choose full scan instead of index
- Extra columns make index less selective

Example Impact:

```
-- BAD: 20 columns transferred, only need 3
SELECT * FROM Employee WHERE ID = 123;
-- Transfers: ID, Name, Email, Phone, Address, Salary, ... (all 20 columns)

-- GOOD: Only needed columns
SELECT ID, Name, Email FROM Employee WHERE ID = 123;
-- Transfers: Only 3 columns
```

Best Practice:

```
-- ALWAYS explicitly list columns
SELECT EmployeeID, Name, Email, Salary
FROM Employee
WHERE DepartmentID = 10;
```

* Why is **UNION ALL** generally faster than **UNION**?

UNION: Removes duplicates from combined result sets **UNION ALL:** Keeps all rows, no deduplication

Why **UNION ALL** is Faster:

1. No Deduplication Cost:

- **UNION** must identify and remove duplicates
- Requires sorting or hashing all rows
- Extra processing step
- **UNION ALL** skips this entirely

2. Sorting Overhead:

- **UNION** often performs sort ($O(n \log n)$)
- **UNION ALL** combines ($O(n)$)
- Sorting adds significant time for large datasets

3. Memory Usage:

- **UNION** needs extra memory for deduplication
- **UNION ALL** minimal extra memory

Example Performance Impact:

```
-- Slow: UNION deduplicates
SELECT ID, Name FROM Employees WHERE DepartmentID = 1
UNION
SELECT ID, Name FROM Employees WHERE DepartmentID = 2;
-- Time: 500ms (includes deduplication)

-- Fast: UNION ALL no deduplication
SELECT ID, Name FROM Employees WHERE DepartmentID = 1
UNION ALL
SELECT ID, Name FROM Employees WHERE DepartmentID = 2;
-- Time: 50ms (just concatenation)
```

When to Use:

- **UNION:** When you actually need to remove duplicates
- **UNION ALL:** If duplicates acceptable (or impossible), always faster
- **Best Practice:** Use UNION ALL unless deduplication needed

Benchmark (1M rows each):

Operation	Time	Memory
UNION	2000ms	500MB
UNION ALL	200ms	50MB

* Why is **ORDER BY** not guaranteed unless explicitly specified?

SQL Doesn't Guarantee Order:

- SQL returns unordered result sets by default
- Row order depends on storage and query optimizer
- Same query can return rows in different order on different runs
- Database doesn't maintain row insertion order

Why No Guaranteed Order:

1. **Relational Model:** SQL is set-based, not sequence-based

- Sets have no inherent order
- SQL follows relational theory

2. **Optimizer Decisions:** Query plan changes affect order

- Different index strategy = different order
- Parallel processing = unpredictable order
- Statistics changes = different plan

3. **Performance:** Guaranteeing order adds cost

- Would require explicit sort
- Every query slower
- Defeats optimization purpose

Example:

```
-- NO GUARANTEE on row order
SELECT * FROM Employee;
-- First run:  John, Jane, Bob
-- Second run: Bob, John, Jane (different order!)

-- GUARANTEED order
SELECT * FROM Employee ORDER BY Name;
-- Always: Bob, Jane, John (alphabetical, guaranteed)
```

Important: If row order matters, ALWAYS use ORDER BY

Performance Note:

- Without ORDER BY: No sort, very fast
- With ORDER BY: Requires sort, slower
- Use ORDER BY only if order truly needed

* **Why is **DISTINCT** considered an expensive operation on large datasets?**

DISTINCT Overhead:

- Must identify and remove duplicate rows
- Expensive operation on large datasets
- Performance degrades significantly with scale

Why DISTINCT is Expensive:

1. Full Scan Requirement:

- Must examine all rows
- Cannot skip rows
- $O(n)$ complexity minimum

2. Deduplication Methods (All Expensive):

- **Sorting:** $O(n \log n)$ - sorts all rows to find duplicates
- **Hashing:** $O(n)$ - builds hash table, uses memory
- **Nested Loops:** $O(n^2)$ - compares all pairs

3. Memory Intensive:

- Hash table holds all unique rows
- Large datasets need massive memory
- Can cause memory overflow

4. Data Transfer:

- Full row set must be processed
- Cannot use partial index
- All columns analyzed

Performance Example:

```
-- Fast: 1 million rows, returns quickly without DISTINCT
SELECT DepartmentID FROM Employee;
-- Time: 100ms, Memory: 10MB

-- Slow: Same 1 million rows, DISTINCT deduplicates
SELECT DISTINCT DepartmentID FROM Employee;
-- Time: 2000ms, Memory: 500MB
-- 20x slower! Because DISTINCT sorts/hashes all rows
```

Alternatives to DISTINCT:

1. GROUP BY (if aggregation needed):

```
-- DISTINCT
SELECT DISTINCT DepartmentID FROM Employee;

-- GROUP BY (if also counting)
SELECT DepartmentID, COUNT(*) FROM Employee GROUP BY DepartmentID;
```

2. EXISTS (if checking existence):

```
-- DISTINCT
SELECT DISTINCT c.CustomerID FROM Customers c JOIN Orders o ON c.ID =
o.CustomerID;

-- EXISTS (faster, no duplicates)
SELECT c.* FROM Customers c WHERE EXISTS (SELECT 1 FROM Orders WHERE CustomerID
= c.ID);
```

Best Practice:

- Avoid DISTINCT on large datasets
- Use GROUP BY or EXISTS instead
- Filter early in query to reduce row count
- Fix data design if duplicates expected

* Why is normalization important, and when should denormalization be considered?

Normalization: Organizing data to minimize redundancy and improve integrity.

Why Normalization Important:

1. Eliminates Data Redundancy:

- Stores data once, referenced many times
- Saves storage space
- Example: Store department once, reference by ID

2. Prevents Data Anomalies:

- **Insertion Anomaly:** Cannot insert department without employee
- **Update Anomaly:** Change department name requires updating all referencing rows
- **Deletion Anomaly:** Deleting employee loses department info
- Normalization prevents these

3. Maintains Data Integrity:

- Constraints enforce consistency
- Foreign keys prevent orphaned records
- Unique constraints prevent duplicates

4. Simplifies Maintenance:

- Changes needed in one place only
- Easier to understand structure
- Cleaner schema

Normalization Levels:

- **1NF:** Eliminate repeating groups
- **2NF:** Remove partial dependencies
- **3NF:** Remove transitive dependencies
- **BCNF:** Handle edge cases

When to Denormalize:

1. Read-Heavy Workloads:

- Queries require expensive JOINS
- Denormalization reduces joins needed
- Example: Store frequently joined data together

2. Performance Critical:

- JOIN performance unacceptable
- Aggregates calculated frequently
- Example: Pre-calculate and store totals

3. Real-Time Analytics:

- Dashboards need instant results
- Aggregation expensive
- Denormalize to materialized view

4. Reporting Databases:

- Data warehouse often denormalized
- Optimized for analysis, not transactions

Example - Normalization vs Denormalization:

Normalized:

```
Employee (ID, Name, DepartmentID)
Department (ID, Name, Location)
-- Query: 2-table JOIN to get employee with department info
```

Denormalized:

```
Employee (ID, Name, DepartmentID, DepartmentName, Location)
-- Query: Single table SELECT (faster)
-- Trade-off: Department data duplicated, harder to update
```

Decision Framework:

- **Start Normalized:** Easier to maintain
- **Denormalize Selectively:** Only if proven performance issue
- **Document Denormalization:** Keep notes on why
- **Monitor Trade-offs:** Ensure performance gain worth consistency cost

* Why is the **EXPLAIN** or execution plan one of the most important tools for SQL performance tuning?

EXPLAIN/Execution Plan: Shows step-by-step how database executes query.

Why It's Critical:

1. Visualize Query Execution:

- See join order
- Identify index usage
- Spot full table scans
- Show data flow

2. Identify Performance Bottlenecks:

- Which step is slow
- Which table causes issue
- Where optimization needed
- Actual vs estimated rows

3. Diagnose Problems:

- Missing index? EXPLAIN shows scan
- Bad join order? EXPLAIN shows sequence
- Correlated subquery? EXPLAIN shows repetition
- Data type mismatch? EXPLAIN shows type conversion

4. Validate Optimizations:

- Before/after index creation
- Query rewrite effectiveness
- Statistics update impact

Example Usage:

SQL Server:

```
SET STATISTICS IO ON;  
SET STATISTICS TIME ON;  
  
SELECT * FROM Employee WHERE Salary > 50000;  
-- Shows: Logical reads, physical reads, CPU time, elapsed time
```

MySQL:

```
EXPLAIN EXTENDED
SELECT e.Name, d.DepartmentName
FROM Employee e
INNER JOIN Department d ON e.DepartmentID = d.ID
WHERE e.Salary > 50000;

-- Shows: Table, type (ALL/ref/eq_ref), keys used, rows examined
```

Reading Execution Plan:

- **Type ALL:** Full table scan (bad, use index)
- **Type ref:** Index seek (good)
- **Type const:** Single row (excellent)
- **Rows:** Estimated rows examined (should be small)
- **Extra:** Using index, using filesort, etc.

What to Look For:

- ✓ Index seeks, not full scans
- ✓ Correct join order
- ✓ Reasonable row estimates
- ✓ No type conversions
- ✓ No unnecessary sorting
- ✓ Parallel execution (if beneficial)
- X Full table scans on large tables
- X Nested loops with many rows
- X Type conversions in WHERE
- X Unexpected sorts/aggregations
- X High row estimates vs actual

Best Practice:

- Check EXPLAIN before deploying queries
- Compare plan changes when performance changes
- Use execution plans to guide index strategy
- Never assume query is efficient without verification

* Why is **EXISTS** often preferred over **IN** for large subqueries?

Reason: Semi-Join vs Full Evaluation

IN Subquery (Less Efficient):

- Evaluates entire subquery result set
- Creates full list of values
- Checks if value IN that list
- Must process all subquery rows

EXISTS Subquery (More Efficient):

- Stops after finding first match
- Returns TRUE/FALSE
- No full list created
- Semi-join optimization

Performance Difference - Large Datasets:

```
-- IN: Returns list [1, 2, 3, ... 100000]
SELECT * FROM Customers
WHERE CustomerID IN (
    SELECT CustomerID FROM Orders -- Builds full list!
);
-- Time: 2000ms (processes all 100K orders)

-- EXISTS: Returns TRUE/FALSE immediately
SELECT * FROM Customers c
WHERE EXISTS (
    SELECT 1 FROM Orders o
    WHERE o.CustomerID = c.CustomerID
); -- Stops at first match
-- Time: 200ms (10x faster!)
```

Why EXISTS Faster:

1. **Early Termination:** Stops after first match
2. **Less Memory:** Doesn't build value list
3. **Correlated Query:** Can use outer row values
4. **Semi-Join:** Optimizer optimizes EXISTS better

NULL Handling:

- IN with NULL: Can cause unexpected results
- EXISTS: Handles NULL correctly

```
-- IN + NULL can be problematic:
SELECT * FROM Customers
WHERE CustomerID NOT IN (
    SELECT CustomerID FROM Orders -- If NULL in list, no rows!
);

-- EXISTS + NULL always works:
SELECT * FROM Customers c
WHERE NOT EXISTS (
    SELECT 1 FROM Orders WHERE CustomerID = c.CustomerID
);
```

When to Use:

- **EXISTS:** Large subqueries, NOT IN, correlated queries
- **IN:** Small subqueries, non-correlated, simple lists

Benchmark (100K rows):

Subquery Type	Time	Memory
IN (all results)	2000ms	500MB
EXISTS (first match)	200ms	50MB

* Why are statistics important for the SQL query optimizer?

Statistics: Metadata about table and column distribution used by optimizer to choose execution plan.

What Statistics Track:

- Column value distribution

- Number of distinct values
- NULL value count
- Data density
- Index usage frequency

Why Statistics Critical:

1. Cardinality Estimation:

- Optimizer estimates rows returned for each operation
- Used to choose execution strategy
- Without statistics, guesses (usually wrong)
- Wrong estimate → wrong plan

2. Index Selection:

- Statistics show if index effective
- High selectivity → Use index
- Low selectivity → Full scan might be better
- Without stats, optimizer blind

3. Join Order Optimization:

- Optimizer chooses join order based on row counts
- Incorrect estimate → Cartesian explosion
- Example: Join 1M rows with 1M rows (bad order)

4. Cost-Based Decisions:

- All cost calculations based on statistics
- CPU cost, I/O cost, memory
- Outdated stats → Inaccurate costs

Example - Stale Statistics Impact:

```
-- Statistics say: Status has 50% NULL, 50% NOT NULL
SELECT * FROM Employee WHERE Status IS NOT NULL;

-- Optimizer chooses: Full scan (thinks 50% of 1M = 500K rows)

-- BUT: Actually 99% NOT NULL = 990K rows
-- Better plan: Index seek (if index on Status)

-- After UPDATE STATISTICS:
SELECT * FROM Employee WHERE Status IS NOT NULL;
-- Now optimizer correctly chooses index
```

Maintaining Statistics:

SQL Server:

```
UPDATE STATISTICS TableName;
UPDATE STATISTICS TableName IndexName;
CREATE STATISTICS stat_name ON TableName(ColumnName);
```

MySQL:

```
ANALYZE TABLE TableName;
ANALYZE TABLE TableName1, TableName2, TableName3;
```

PostgreSQL:

```
ANALYZE TableName;
ANALYZE TableName(ColumnName);
```

When to Update Statistics:

- After data bulk changes (INSERT, DELETE, UPDATE large volumes)
- After index creation
- Query performance suddenly degrades
- Scheduled maintenance (weekly/monthly)
- Before major reporting runs

Best Practice:

```
-- Update statistics for most-changed table:
UPDATE STATISTICS Employee;
UPDATE STATISTICS Orders;

-- Or enable auto-update:
SET AUTO_UPDATE_STATISTICS ON; -- SQL Server
SET GLOBAL stats_auto_recalc='ON'; -- MySQL (Percona)
```

Impact of Outdated Statistics:

Issue	Effect	Impact
Stale stats	Wrong row estimates	Suboptimal plan
Missing stats	No estimates	Random plan
Skewed distribution	Wrong index choice	Slow queries

Key Takeaway: Statistics = Foundation of optimization. Keep them fresh!

* Why do databases use B-Tree indexes by default instead of other data structures?

B-Tree: Balanced tree structure used by most databases for indexes.

Why B-Trees Chosen:

1. Balanced Structure:

- Self-balancing: Maintains height consistency
- $O(\log n)$ search, insert, delete (guaranteed)
- No worst-case degradation

2. Disk I/O Optimization:

- Sorted order minimizes disk accesses
- Retrieves multiple values per disk page

- Range queries efficiently scan consecutive
- Example: All orders in date range, one pass

3. Multi-Level Indexing:

- Branch nodes point to other nodes
- Leaf nodes contain actual data/pointers
- Few levels needed even for billions of rows
- Example: 100M rows need only ~4 levels

4. Range Query Efficiency:

- Hash tables poor for ranges
- B-Tree optimized for: BETWEEN, >, <, IN
- Sorted structure enables binary search

5. Sequential Access:

- Linked leaf nodes support NEXT/PREVIOUS
- Efficient ORDER BY, GROUP BY
- Perfect for time-series data

B-Tree Variants:

- **B+ Tree:** Improved, data in leaf nodes only (most used)
- **B Tree:*** More balanced
- **R-Tree:** Spatial data (GIS)
- **Suffix Tree:** String searching

Examples from Databases:

- **MySQL/InnoDB:** B+ Tree
- **PostgreSQL:** B-Tree (default)
- **SQL Server:** B-Tree
- **Oracle:** B-Tree (default)

When Other Structures Used:

- **Hash Index:** Equality searches only (rare)
- **Bitmap:** Low cardinality columns (gender, status)
- **Full-text:** Text searching (LIKE queries)

- **Spatial:** Geographic coordinates (GIS)

B-Tree Structure Example:

```

Level 0 (Root):           [40]
                        /   \
Level 1:         [20, 30]   [50, 60]
                  / | \     / | \
Level 2 (Leaf): 10 20 30 40 50 60 70 80
  
```

Search for 35:

- Start at 40: $35 < 40$, go left
- Check 20, 30: $35 > 30$, go to middle branch
- Check leaf: 35 not found, but know range

Performance Characteristics:

- Lookup: $O(\log n)$ ~20 seeks for 1 billion rows
- Insert: $O(\log n)$ + rebalancing
- Range: Sequential scan after binary search
- Space: ~20% overhead for tree structure

*** Why does the order of conditions in a WHERE clause usually not affect query performance, even though many developers believe it does?**

Common Belief: Developers think order of WHERE conditions matters for performance.

Truth: Modern query optimizers reorder conditions (mostly) independent of written order.

Why Order Doesn't Matter (Usually):

1. Query Optimizer Reordering:

- Optimizer analyzes all conditions
- Reorders based on cost estimates
- Selectivity, index availability, statistics
- Optimizer decides best order, ignores written order

2. Cost-Based Optimization:

- Most selective condition first
- Filtered early to reduce row count
- Optimizer calculates cost for all orders
- Chooses optimal sequence

3. Index Usage:

- Index determines execution, not condition order
- If index on column, that condition evaluated early
- Condition order ignored

Example - Order Doesn't Matter:

```
-- Query 1: General condition first
SELECT * FROM Employee
WHERE Status = 'Active'
AND DepartmentID = 10
AND Salary > 50000;

-- Query 2: Specific condition first
SELECT * FROM Employee
WHERE Salary > 50000
AND DepartmentID = 10
AND Status = 'Active';

-- Both generate SAME execution plan!
-- Optimizer reorders to most selective first
-- Time: Identical ~100ms
```

When Order MIGHT Matter:

1. Short-Circuit Evaluation (Rare):

- Some DBs evaluate left to right
- If first condition false, skip rest
- Example: `WHERE 0 = 1 AND ExpensiveFunction()`
- But: Optimizer usually prevents this

2. Correlated Subqueries:

- Must put outer reference first

- Example: `WHERE e.DeptID = d.ID AND d.DeptID = 10`
- Order affects subquery execution

3. Column Statistics Missing:

- Optimizer can't estimate cost
- Falls back to written order (rare)
- But: Should update statistics

Myth Examples (Order Doesn't Matter):

```
-- MYTH: Expensive function last is slower
WHERE Cheap_Column = 1 AND ExpensiveFunction() > 100;

-- REALITY: Optimizer evaluates Cheap_Column first
-- (most selective), skips ExpensiveFunction() often

-- MYTH: Most selective first mandatory
WHERE Salary > 50000 AND DepartmentID = 10;
WHERE DepartmentID = 10 AND Salary > 50000;
-- Both same performance
```

Best Practice:

- Write conditions in logical order (readability)
- Don't worry about performance order
- Trust optimizer to reorder
- If performance issue, check execution plan
- Optimize via indexes, not condition order

Why This Myth Exists:

- Early DBs evaluated left-to-right
- Developer habits from 1990s/2000s
- Some developers experienced early optimization
- Misconception persisted

Focus On What Matters: ✓ Index selectivity (not condition order) ✓ WHERE conditions early (before aggregates) ✓ Reduce result set before GROUP BY ✓ Update statistics regularly ✓ Write clear, maintainable SQL